

Project

Mobile Application Development

dr inż. Radosław Maciaszczyk

Faculty of Computer Science

West Pomeranian University of Technology, Szczecin

Project – Class 2

Deadline: 7 April 2026

Design the **application views** (screens) for the main functions of your planned application and submit them as a **PDF file**.

The views should cover all key screens defined in your Class 1 description and reflect the navigation flow between them.

Task Requirements

1. Screen list

Provide a list of all designed screens with a short description of the purpose of each one.

2. Wireframes / mockups

For each screen, include a visual representation showing the layout of UI elements (buttons, text fields, lists, maps, etc.).

The level of fidelity is flexible – from hand-drawn sketches to high-fidelity mockups.

3. Navigation diagram

Include a simple diagram (flowchart or arrow diagram) showing how the user navigates between screens (which action on which screen leads where).

4. Mapping to functional requirements

For each screen, indicate which functional requirement(s) from Class 1 are addressed by that screen.

Recommended Tools

Tool	Notes
Figma	Free, browser-based; supports components, prototyping, and sharing. Available at https://www.figma.com/design/
Android Studio	Create an empty Compose project and implement the screens directly in Kotlin / Jetpack Compose with placeholder data. Export screenshots for the PDF.
Other tools	Balsamiq, Sketch, Pencil, draw.io, or even hand-drawn scans are also acceptable.

Tip: If you choose Android Studio, using **Jetpack Compose Previews** (`@Preview` annotation) is the fastest way to iterate on your layouts without running the app on a device or emulator.

Submission

- Submit a single **PDF file** containing all screens and the navigation diagram.
- If Figma is used, also include a **shareable link** to the Figma project (set access to *Anyone with the link – can view*).
- If Android Studio is used, include the **relevant source files** in your GitHub repository in a **design/** or **wireframes/** folder.

Note: The designs do not have to be final – they may evolve during development. The goal at this stage is to demonstrate a clear understanding of the application structure and user flow.

Project – Class 2

Sample description

Application: WeatherNow (Medium level – continued from Class 1)

1. Screen list

Screen	Purpose
Home	Displays current weather for the detected location
Search	Allows the user to search for a city by name
Detail	Shows full weather details and 5-day forecast for a location
History	Lists the last 10 searched locations
Favourites	Lists user-marked favourite locations

2. Wireframes

Wireframes created in **Figma**.

Shareable link: <https://www.figma.com/design/...>

Screenshots of all five screens are included on the following pages of this PDF.

3. Navigation diagram

From	Action	To
Home	tap Search icon	Search
Home	tap History tab	History
Home	tap Favourites tab	Favourites
Search	select a result	Detail
History	tap a location	Detail
Favourites	tap a location	Detail
Detail	tap star icon	(marks as favourite)

4. Mapping to functional requirements

Screen	Functional requirements covered
Home	Check current weather for current location (GPS); auto-refresh on open
Search	Search for weather by city name
Detail	Display temperature, humidity, wind speed, weather icon, 5-day forecast; show location on Google Maps minimap
History	Show history of last 10 searched locations
Favourites	Mark/access favourite locations from the home screen

Appendix: Figma and Jetpack Compose – Integration Tools

Note: Google Relay (the former official Figma–Compose bridge) was **sunset on 30 April 2025** and is no longer supported. Do not start new projects with it.

The tools below represent the current (2025–2026) recommended approaches for transferring a Figma design into a Jetpack Compose Android project.

1. Figma Code Connect (Official – Dev Mode)

What it does: Links your existing Kotlin/Compose components to the corresponding Figma components so that **Figma Dev Mode** shows real code snippets instead of auto-generated placeholders. It does *not* generate code – it connects already-written code to the design.

Setup in brief:

1. Install the CLI: `npm install -global @figma/code-connect@latest`
2. Add the Code Connect Gradle dependency to `build.gradle.kts`
3. Generate a Figma personal access token (scopes: *Code Connect: Write* and *File content: Read*)
4. Run the interactive wizard: `npx figma connect --token=YOUR_TOKEN`
5. Annotate your Composables with `@FigmaConnect`, `@FigmaProperty`, `@FigmaVariant`
6. Publish: `npx figma connect publish --token=YOUR_TOKEN`

Official documentation: <https://developers.figma.com/docs/code-connect/compose/>

2. Builder.io Visual Copilot (AI – Code Generation)

What it does: A Figma plugin that converts a selected screen or component into production-ready Jetpack Compose Kotlin code in one click. Always review and refactor the generated output.

1. Install the *Visual Copilot* plugin from the Figma Community.
2. Select the frame or component to export.
3. Click **Export to Compose** – the plugin generates Kotlin Composables.
4. Paste into Android Studio, review, and integrate with your architecture.

Guide: <https://www.builder.io/blog/convert-figma-to-jetpack-compose>

3. Google AI Studio / Gemini (AI – Screenshot to Code)

Export a screenshot of your Figma screen, upload it to Google AI Studio, and prompt Gemini to generate Jetpack Compose code from the image. Useful for quickly scaffolding simple layouts.

Step-by-step guide: <https://medium.com/@ahmetbostanciklioglu/transforming-figma-ui-design>

4. Community Figma Plugins

Plugin	Figma Community URL
Figma to Compose	https://www.figma.com/community/plugin/856651176156241740
Figma2Android	https://www.figma.com/community/plugin/1030354617366499719
CodeTea (multi-framework)	https://www.figma.com/community/plugin/1299643423790238613

Recommendation: Community plugins vary in code quality. Treat their output as a starting point, not finished code. For anything beyond simple static layouts, manual implementation remains necessary.

Pricing

Tool	Free tier	Paid from	Students
Figma (basic)	Yes (2 editors, 3 files)	\$12/seat/mo	Free Professional via figma.com/education
Figma Dev Mode	No	included in Professional seat	Included in student plan
Figma Code Connect	No	Organization (\$45/seat/mo)	Not available on student plan
Builder.io Visual Copilot	Yes (75 cred- its/mo, 25/day cap)	\$19/user/mo	No student plan; free tier sufficient for coursework
Google AI Studio (Gemini)	Yes (fully free)	pay-per-token (optional)	Free for everyone; no special plan needed

Student tip: Register on <https://www.figma.com/education/> with your university e-mail to get a **free Figma Professional plan** (including Dev Mode) valid for 2 years. No credit card required. Figma Code Connect requires a paid Organization plan and is **not needed** for this course.

Recommended Workflow

#	Step	Action
1	Design	Create/finalize screens in Figma using components and auto-layout
2	Dev Mode	Use Figma Dev Mode to inspect spacing, colours, and component properties
3	Boilerplate	Use Builder.io or Gemini to generate initial Composables
4	Implement	Manually add navigation, state, repositories, and business logic
5	Review	Refactor generated code for correctness, performance, and architecture fit